

Project structure & routing

COMPLETE LEARNING GUIDE

What You'll Learn

A backend project's folder structure and routing scheme are the first things a new developer sees, and a messy one is one of the strongest predictors of a codebase that's expensive to change. This guide covers how to organize a growing server application — separating routes, controllers, services, and data-access code into distinct layers — and how to design URL routes that stay predictable as the number of endpoints grows into the dozens or hundreds. It focuses on Express's Router() as the concrete mechanic, since modular routing is where most real structural decisions get made, while also covering how Django's app-based layout and Spring Boot's package organization solve the same underlying problem. By the end you should be able to look at an unfamiliar backend codebase and immediately locate where a given endpoint's logic lives.

1. Why Structure Matters More Than It Seems Early On

A five-route prototype can live entirely in one file with no ill effects, which is exactly why bad habits form early: it's easy to keep adding routes to that same file until it's two thousand lines long and nobody wants to touch it. The cost of poor structure isn't visible until the project has grown past the point where restructuring is cheap. The fix is to adopt a layered structure before it's strictly necessary — separating 'what URL maps to what handler' (routing) from 'what business logic runs' (controllers/services) from 'how data is read and written' (models/repositories) — so that each concern can be found, tested, and changed independently.

2. The Layered Architecture Pattern

Most mature backend codebases converge on some version of the same three-to-four layer split, regardless of framework or language. Routes/controllers receive the HTTP request, extract and validate input, and return an HTTP response — they should contain almost no business logic. Services (or 'use cases') contain the actual business rules, orchestrating calls to one or more data sources, independent of HTTP entirely. Repositories (or models/DAOs) are the only layer that talks to the database, isolating SQL or ORM calls from the rest of the app. This separation means a business rule can be unit-tested without spinning up an HTTP server, and a database can be swapped without touching controller code.

- Routes/Controllers: parse the request, call a service, shape the response — no business logic.
- Services: business rules and orchestration, framework-agnostic and easily unit-testable.
- Repositories/Models: the only layer that issues database queries.
- This split is often called MVC (Model-View-Controller) or a 'layered' / 'clean' architecture depending on the ecosystem's vocabulary.

3. Modular Routing in Express with Router()

Express's `express.Router()` creates a mini, mountable route handler — effectively a mini-app for one resource — that a parent app can mount at a URL prefix with `app.use()`. This is the standard way to avoid one giant routes file: each resource (users, orders, products) gets its own router module, which is then composed together in the main app file. Because the router only knows its routes relative to its mount point, a resource's routes can be moved to a different URL prefix by changing a single line in the parent file.

JAVASCRIPT

```
// routes/users.js
const express = require("express");
const router = express.Router();
const usersController = require("../controllers/usersController");

router.get("/", usersController.listUsers);
router.get("/:id", usersController.getUser);
router.post("/", usersController.createUser);
router.put("/:id", usersController.updateUser);
router.delete("/:id", usersController.deleteUser);

module.exports = router;

// app.js
const usersRouter = require("./routes/users");
app.use("/api/users", usersRouter);
```

4. Route Parameters, Query Strings, and Naming Consistency

Express distinguishes between route parameters (`:id`, part of the URL path, used to identify a specific resource) and query strings (`?sort=name`, used for optional filtering, sorting, or pagination). A common structural bug is naming the same conceptual parameter differently across routes — `:id` in one file, `:userId` in another — which makes middleware and helper functions that read `req.params` fragile and inconsistent. Establishing a naming convention early (e.g., always `:resourceId`, never bare `:id`, once a route can be nested) pays off as soon as two resources are nested under one another.

- `req.params` holds route/path parameters: `GET /api/users/:id !' req.params.id`.

- `req.query` holds the query string: `GET /api/users?role=admin&sort=name !` `req.query.role`, `req.query.sort`.
- Keep parameter names consistent across sibling routes so shared middleware can rely on a fixed key.
- Nested resource routes (e.g. `/api/orders/:orderId/items/:itemId`) need distinct names for each level's identifier.

5. Feature Folders vs. Layer Folders

There are two common ways to organize files past the routing layer: 'group by technical layer' (a top-level `controllers/`, `services/`, `models/` folder each containing files for every resource) or 'group by feature' (a top-level `orders/` folder containing that resource's controller, service, and model together). Layer-first organization is easy to start with and matches how many tutorials teach the pattern, but it means implementing or understanding one feature requires jumping between three or four distant folders. Feature-first organization keeps everything related to one concern together, and scales better once the number of resources grows past a handful — most large real-world codebases migrate toward it over time.

6. How Django and Spring Boot Solve the Same Problem

Django enforces feature-style organization structurally: each 'app' (a Python package, e.g. `orders/`) contains its own `models.py`, `views.py`, `urls.py`, and `serializers.py`, and a project is a collection of such apps wired together in a root `URLconf`. Spring Boot doesn't mandate a layout, but idiomatic projects package by feature (`com.example.orders` containing `OrderController`, `OrderService`, `OrderRepository`) rather than by layer, for the same reasons Express codebases eventually migrate that way. In both ecosystems, the underlying lesson is identical to Express's: keep the code for one resource physically close together, and keep routing thin.

7. Route Ordering and Catch-All Handlers

Express matches routes in the order they're registered, and matching stops at the first route (or middleware) that handles the request — this makes ordering a real source of bugs. A wildcard or catch-all route (e.g. a 404 handler using `app.use((req, res) => {...})`) must be registered last, after every specific route, or it will swallow requests meant for routes defined after it. Similarly, a more specific static route like `/api/users/me` should generally be registered before a parameterized one like `/api/users/:id` if both could match the same literal path, to avoid 'me' being interpreted as an `:id` value.

- Express matches top-to-bottom; the first matching route/middleware wins.
- Register a 404/catch-all handler as the very last `app.use()` call.
- Put specific literal routes (`/users/me`) before parameterized ones (`/users/:id`) when both could match.
- Mounting order of routers (`app.use('/api/orders', ...)`) also matters when prefixes could overlap.

Common Pitfalls

- Putting business logic (validation, calculations, orchestration) directly inside route handlers instead of a separate service layer, making it untestable without an HTTP server.
- Using inconsistent route parameter names (:id vs :userId vs :user_id) across sibling routes, breaking shared middleware that expects one convention.
- Registering a catch-all 404 handler before other routes, silently swallowing requests meant for routes defined later in the file.
- Letting a single routes file grow to hundreds of lines instead of splitting by resource with `express.Router()` once past a handful of endpoints.
- Deeply nesting resource routes (more than two levels, e.g. `/api/companies/:id/departments/:id/employees/:id/tasks/:id`) instead of flattening with query params or separate top-level lookups.
- Mixing HTTP verbs inconsistently for the same operation across resources — DELETE on one resource, POST `/delete-x` on another.

Key Takeaways

- Separate routing (what URL maps where), controllers (parse/respond), services (business logic), and repositories (data access) into distinct layers.
- `express.Router()` is the standard tool for splitting routes by resource instead of keeping one monolithic route file.
- Route parameters (:id) identify a specific resource; query strings (?sort=) filter or shape a collection — don't conflate the two.
- Express matches routes in registration order — specific routes and catch-alls must be ordered deliberately.
- Feature-first folder organization (group by resource) scales better than layer-first (group by technical role) as a project grows.
- Django and Spring Boot enforce or encourage the same feature-based grouping Express projects eventually converge on by convention.

Learn More

Express.js — Routing Guide

<https://expressjs.com/en/guide/routing.html>

MDN — Django Tutorial: URLs and Views

https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/Django/urls

Baeldung — Spring Boot Project Structure

<https://www.baeldung.com/spring-boot-application-structure>